

M5 Agentic AI - Customer Service Agent

1. Introduction

As Andrew explained in the lecture, *planning with code execution* means letting the LLM **write code that becomes the plan itself**.

Compared to plain-text or JSON-based plans, this approach is more expressive and flexible: the code not only documents the steps but can also execute them directly.

In this lab, you will implement this design pattern in practice.

Instead of asking the LLM to output a plan in JSON format and then manually executing each step, we will allow it to **write Python code** that directly captures multiple steps of a plan. By executing this code, we can carry out complex queries automatically.

To make things concrete, we simulate a **sunglasses store** with an **inventory** of products and a set of **transactions** (sales, returns, balance updates). This example shows how the LLM can generate code to query or update records, demonstrating the flexibility of this pattern.

1.1 Lab Overview

We will:

1. Create simple **inventory** and **transaction** datasets.
2. Build a **schema block** describing the data.
3. Prompt the LLM to **write a plan as Python code** (with comments explaining each step).
4. Execute the code in a sandbox to obtain the answer.

1.2 Learning Outcomes

By the end of this lab, you will be able to:

- **Explain** why letting the model write code (instead of JSON or plain text plans) enables richer, more flexible planning.
- **Prompt** an LLM to produce Python code with step-by-step comments that both documents and executes the plan.
- **Run** the generated code safely in a sandbox and interpret the results.

This illustrates how *Code as Action* can outperform brittle tool chains and JSON-based planning approaches.

2. Setup

```
In [1]: # ==== Imports ====
from __future__ import annotations
import json
from dotenv import load_dotenv
from openai import OpenAI
import re, io, sys, traceback, json
from typing import Any, Dict, Optional
from tinydb import Query, where

# Utility modules
import utils      # helper functions for prompting/printing
import inv_utils  # functions for inventory, transactions, schema building,

load_dotenv()
client = OpenAI()
```

In the `inv_utils` module, we have functions like:

- `create_inventory()` – builds the sunglasses inventory.
- `create_transactions()` – builds the initial transaction log.
- `seed_db()` – loads both inventory and transactions into a JSON-backed store.
- `build_schema_block()` – generates a schema description used in the prompt.
- Helpers like `get_current_balance()` and `next_transaction_id()` – let the LLM handle consistent updates across inventory and transactions.

2.1 Create Example Tables

We will now create two small tables for the sunglasses store simulation, using **TinyDB** — a lightweight document-oriented database written in pure Python.

TinyDB stores data as JSON documents and is well-suited for small applications or prototypes, since it requires no server setup and allows you to query and update data easily.

The two tables are:

- **inventory_tbl** : contains product details such as name, item ID, description, quantity in stock, and price.
- **transactions_tbl** : starts with an opening balance and will later track purchases, returns, and adjustments.

You will generate these tables using helper functions in `inv_utils`, and then preview the first few rows below.

```
In [2]: db, inventory_tbl, transactions_tbl = inv_utils.seed_db()
```

Now, you can inspect the records in each table by printing them as formatted JSON:

```
In [3]: utils.print_html(json.dumps(inventory_tbl.all(), indent=2), title="Inventory")
        utils.print_html(json.dumps(transactions_tbl.all(), indent=2), title="Transactions")
```

Inventory Table

```
[
  {
    "item_id": "SG001",
    "name": "Aviator",
    "description": "Originally designed for pilots, these teardrop-shaped lenses with thin metal frames offer timeless appeal. The large lenses provide excellent coverage while the lightweight construction ensures comfort during long wear.",
    "quantity_in_stock": 23,
    "price": 80
  },
  {
    "item_id": "SG002",
    "name": "Wayfarer",
    "description": "Featuring thick, angular frames that make a statement, these sunglasses combine retro charm with modern edge. The rectangular lenses and sturdy acetate construction create a confident look.",
    "quantity_in_stock": 6,
    "price": 95
  },
  {
    "item_id": "SG003",
    "name": "Mystique",
    "description": "Inspired by 1950s glamour, these frames sweep upward at the outer corners to create an elegant, feminine silhouette. The subtle curves and often embellished temples add sophistication to any outfit.",
    "quantity_in_stock": 3,
    "price": 70
  },
  {
    "item_id": "SG004",
    "name": "Sport",
    "description": "Designed for active lifestyles, these wraparound sunglasses feature a single curved lens that provides maximum coverage and wind protection. The lightweight, flexible frames include rubber grips.",
    "quantity_in_stock": 11,
    "price": 110
  },
  {
    "item_id": "SG005",
    "name": "Classic",
    "description": "Classic round profile with minimalist metal frames, offering a timeless and versatile style that fits both casual and formal wear.",
    "quantity_in_stock": 10,
    "price": 60
  },
  {
    "item_id": "SG006",
```

```

    "name": "Moon",
    "description": "Oversized round style with bold plastic frame
s, evoking retro aesthetics with a modern twist.",
    "quantity_in_stock": 10,
    "price": 120
  }
]

```

Transactions Table

```

[
  {
    "transaction_id": "TXN001",
    "customer_name": "OPENING_BALANCE",
    "transaction_summary": "Daily opening register balance",
    "transaction_amount": 500.0,
    "balance_after_transaction": 500.0,
    "timestamp": "2026-08-05T18:52:21.280982"
  }
]

```

As you can see above, the schemas of each table are as follows:

Inventory Table (`inventory_tbl`)

- **item_id** (string): Unique product identifier (e.g., SG001).
- **name** (string): Style of sunglasses (e.g., Aviator, Round).
- **description** (string): Text description of the product.
- **quantity_in_stock** (int): Current stock available.
- **price** (float): Price in USD.

Transactions Table (`transactions_tbl`)

- **transaction_id** (string): Unique identifier (e.g., TXN001).
- **customer_name** (string): Name of the customer, or `OPENING_BALANCE` for initial entry.
- **transaction_summary** (string): Short description of the transaction.
- **transaction_amount** (float): Amount of money for this transaction.
- **balance_after_transaction** (float): Running balance after applying the transaction.
- **timestamp** (string): ISO-8601 formatted date/time of the transaction.

Planning with Code Execution

2.1. The plan

Once the schema is clear, you'll build the **prompt** that instructs the model to *plan by writing code* and then execute that code. As Andrew emphasized, the code is the plan: the model explains each step in comments, then carries it out. Your prompt below also makes the model self-decide whether the request is read-only or a state change, and it enforces safe execution (no I/O, no network, TinyDB Query only, consistent mutations).

```
In [4]: PROMPT = """You are a senior data assistant. PLAN BY WRITING PYTHON CODE USING THE FOLLOWING DATABASE SCHEMA AND SAMPLES (read-only):

Database Schema & Samples (read-only):
{schema_block}

Execution Environment (already imported/provided):
- Variables: db, inventory_tbl, transactions_tbl # TinyDB Table objects
- Helpers: get_current_balance(tbl) -> float, next_transaction_id(tbl, prefix) -> int
- Natural language: user_request: str # the original user message

PLANNING RULES (critical):
- Derive ALL filters/parameters from user_request (shape/keywords, price ranges, quantities, buy/return intent). Do NOT hard-code values.
- Build TinyDB queries dynamically with Query(). If a constraint isn't in use, use None.
- Be conservative: if intent is ambiguous, do read-only (DRY RUN).

TRANSACTION POLICY (hard):
- Do NOT create aggregated multi-item transactions.
- If the request contains multiple items, create a separate transaction row for each item.
- For each item:
  - compute its own line total (unit_price * qty),
  - insert ONE transaction with that amount,
  - update balance sequentially (balance += line_total),
  - update the item's stock.
- If any requested item lacks sufficient stock, do NOT mutate anything; reply with 'insufficient stock'.

HUMAN RESPONSE REQUIREMENT (hard):
- You MUST set a variable named `answer_text` (type str) with a short, customer-facing message.
- This sentence is the only user-facing message. No dataframes/JSON, no boilerplate.
- If nothing matches, politely say so and offer a nearby alternative (closest item by name/price).

ACTION POLICY:
- If the request clearly asks to change state (buy/purchase/return/restock/adjust stock), set
  ACTION="mutate"; SHOULD_MUTATE=True; perform the change and write a matching message.
  Otherwise:
  ACTION="read"; SHOULD_MUTATE=False; simulate and explain briefly as a data analyst.

FAILURE & EDGE-CASE HANDLING (must implement):
- Do not capture outer variables in Query.test. Pass them as explicit args.
- Always set a short `answer_text`. Also set a string `STATUS` to one of:
  "success", "no_match", "insufficient_stock", "invalid_request", "unsupported_request"
  """
```

- no_match: No items satisfy the filters → suggest the closest in style/price
- insufficient_stock: Item found but stock < requested qty → state available
- invalid_request: Unable to parse essential info (e.g., quantity for a purchase)
- unsupported_intent: The action is outside the store's capabilities → provide alternatives
- In all cases, keep the tone helpful and concise (1–2 sentences). Put technical details in a separate block.

OUTPUT CONTRACT:

- Return ONLY executable Python between these tags (no extra text):
`<execute_python>`
`# your python`
`</execute_python>`

CODE CHECKLIST (follow in code):

- 1) Parse intent & constraints from user_request (regex ok).
- 2) Build TinyDB condition incrementally; query inventory_tbl.
- 3) If mutate: validate stock, update inventory, insert a transaction (new id)
- 4) ALWAYS set:
 - `answer\_text` (human sentence, required),
 - `STATUS` (see list above).
 Also print a brief log to stdout, e.g., "LOG: ACTION=read DRY_RUN=True STATUS=ok"
- 5) Optional: set `answer\_rows` or `answer\_json` if useful, but `answer\_text` is required.

TONE EXAMPLES (for `answer\_text`):

- success: "Yes, we have our Classic sunglasses, a round frame, for \$60."
- no_match: "We don't have round frames under \$100 in stock right now, but we have some other options."
- insufficient_stock: "We only have 1 pair of Classic left; I can reserve that for you."
- invalid_request: "I can help with that—how many pairs would you like to purchase?"
- unsupported_intent: "We can't refurbish frames, but I can suggest similar items."

Constraints:

- Use TinyDB Query for filtering. Standard library imports only if needed.
- Keep code clear and commented with numbered steps.

User request:

```
{question}
"""
```

2.2 From Prompt to Code (Planning in Code)

Let's generate code that **is the plan**.

Instead of asking the model to output a plan in JSON and running it step-by-step with many tiny tools, let's have it **write Python that encodes the whole plan** (e.g., "filter this, then compute that, then update this row"). The function `generate_llm_code`:

1. **Builds a live schema** from `inventory_tbl` and `transactions_tbl` so the model sees real fields, types, and examples.
2. **Formats the prompt** with that schema plus the user's question.
3. **Calls the model** to produce a **plan-with-code** response — typically an `<execute_python>...</execute_python>` block whose body contains the step-by-step logic.

4. Returns the full response (including the plan and the code).

We don't execute anything in this step.

Why this pattern? Let's leverage Python/TinyDB as a rich toolbox the model already "knows," so it can compose multi-step solutions directly in code instead of relying on a growing set of bespoke tools. We'll extract and run the code in a later step.

```
In [5]: # ----- 1) Code generation -----
def generate_llm_code(
    prompt: str,
    *,
    inventory_tbl,
    transactions_tbl,
    model: str = "gpt-4.1-mini",
    temperature: float = 0.2,
) -> str:
    """
    Ask the LLM to produce a plan-with-code response.
    Returns the FULL assistant content (including surrounding text and tags)
    The actual code extraction happens later in execute_generated_code.
    """

    schema_block = inv_utils.build_schema_block(inventory_tbl, transactions_tbl)
    prompt = PROMPT.format(schema_block=schema_block, question=prompt)

    resp = client.chat.completions.create(
        model=model,
        temperature=temperature,
        messages=[
            {
                "role": "system",
                "content": "You write safe, well-commented TinyDB code to handle inventory and transactions data.",
            },
            {"role": "user", "content": prompt},
        ],
    )
    content = resp.choices[0].message.content or ""

    return content
```

2.3 Try a Sample Prompt (Planning-in-Code)

We'll use the same prompt Andrew used in the lecture:

Prompt: "Do you have any round sunglasses in stock that are under \$100?"

Before generating any code, let's manually inspect the TinyDB tables to see if there are truly *round* frames (word-only match) and what their prices look like. Run the next cell to preview the inventory and highlight items that match the word-only "round" filter.

```
In [6]: Item = Query() # Create a Query object to reference fields
```



```

# Search the inventory table for documents where either the description OR the name
# contains the word "round" (case-insensitive). The check is done inline:
# - (v or "") ensures we handle None by converting it to an empty string
# - .lower() normalizes case
# - " round " enforces a crude word boundary (won't match "wraparound")
round_sunglasses = inventory_tbl.search(
    (Item.description.test(lambda v: " round " in ((v or "").lower())) |
     Item.name.test(lambda v: " round " in ((v or "").lower())))
)

# Render the results as formatted JSON in the notebook UI
utils.print_html(json.dumps(round_sunglasses, indent=2), title="Inventory Status: Round Sunglasses")

```

Inventory Status: Round Sunglasses

```

[
  {
    "item_id": "SG005",
    "name": "Classic",
    "description": "Classic round profile with minimalist metal frames, offering a timeless and versatile style that fits both casual and formal wear.",
    "quantity_in_stock": 10,
    "price": 60
  },
  {
    "item_id": "SG006",
    "name": "Moon",
    "description": "Oversized round style with bold plastic frames, evoking retro aesthetics with a modern twist.",
    "quantity_in_stock": 10,
    "price": 120
  }
]

```

Great — we do have round frames available. From our manual inspection, there are two round styles in stock, but only **one** is **under \$100**. Therefore, the item that satisfies the requirement is:

```
{
  "item_id": "SG005",
  "name": "Classic",
  "description": "Classic round profile with minimalist metal
frames, offering a timeless and versatile style that fits both
casual and formal wear.",
  "quantity_in_stock": 10,
  "price": 60
}
```

Now let's ask the model to **generate a plan in code** that answers Andrew's prompt (no execution yet).

```
In [7]: # Andrew's prompt from the lecture
prompt_round = "Do you have any round sunglasses in stock that are under $100"

# Generate the plan-as-code (FULL content; may include <execute_python> tags
full_content_round = generate_llm_code(
    prompt_round,
    inventory_tbl=inventory_tbl,
    transactions_tbl=transactions_tbl,
    model="o4-mini",
    temperature=1.0,
)

# Inspect the LLM's plan + code (no execution here)
utils.print_html(full_content_round, title="Plan with Code (Full Response)")
```

Plan with Code (Full Response)

```

<execute_python>
import re
from tinydb import Query

# 1) Parse intent & constraints from user_request
shape = None
price_min = None
price_max = None

# Extract shape keyword
if re.search(r'\bround\b', user_request, re.IGNORECASE):
    shape = 'round'

# Extract price constraints: under, over, between
m_under = re.search(r'under\s*\$?(\\d+)', user_request, re.IGNORECASE)
if m_under:
    price_max = float(m_under.group(1))

m_over = re.search(r'over\s*\$?(\\d+)', user_request, re.IGNORECASE)
if m_over:
    price_min = float(m_over.group(1))

m_between = re.search(r'between\s*\$?(\\d+)\s*(?:and|to)\s*\$?(\\d+)', user_request, re.IGNORECASE)
if m_between:
    price_min = float(m_between.group(1))
    price_max = float(m_between.group(2))

# 2) Build TinyDB condition incrementally
Product = Query()
cond = (Product.quantity_in_stock > 0)
if price_min is not None:
    cond &= (Product.price >= price_min)
if price_max is not None:
    cond &= (Product.price <= price_max)
if shape:
    # match shape in description or name
    cond &= (
        Product.description.test(lambda d, s=shape: s.lower() in d.lower()) |
        Product.name.test(lambda n, s=shape: s.lower() in n.lower())
    )

# 3) Query inventory_tbl (read-only)
results = inventory_tbl.search(cond)

# 4) Formulate answer and status
if results:
    # pick first matching item

```

```

    item = results[0]
    answer_text = f"Yes, we have our {item['name']} sunglasses, a
round frame, for ${item['price']}."
    STATUS = "success"
else:
    # no matches, suggest closest alternative by price
    # find any in stock with shape if possible
    alt_cond = (Product.quantity_in_stock > 0)
    if shape:
        alt_cond &= (
            Product.description.test(lambda d, s=shape: s.lower()
in d.lower()) |
            Product.name.test(lambda n, s=shape: s.lower() in n.lo
wer())
        )
    alts = inventory_tbl.search(alt_cond)
    if alts:
        # sort by price ascending and pick first above range or ne
arest
        alts_sorted = sorted(alts, key=lambda x: x['price'])
        alt = alts_sorted[0]
        answer_text = (f"We don't have round frames under ${int(pr
ice_max)} in stock right now, "
            f"but our {alt['name']} round frame is avai
lable at ${alt['price']}." )
        else:
            answer_text = "We don't have round sunglasses in stock rig
ht now; can I help you find another style?"
            STATUS = "no_match"

# Log and set action
print(f"LOG: ACTION=read DRY_RUN=True STATUS={STATUS}")
</execute_python>

```

2.4. Define the executor function (run a given plan)

Now we'll define the function that **takes a plan produced by the model and runs it** safely:

- It **accepts either** the full LLM response (with `<execute_python>...`
`</execute_python>`) **or** raw Python code.
- It **extracts** the executable block when needed.
- It runs the code in a **controlled namespace** (TinyDB tables + safe helpers only).
- It captures **stdout**, **errors**, and the model-set answer variables (`answer_text` ,
`answer_rows` , `answer_json`).
- It renders **before/after** table snapshots to make side effects explicit.

This is the "executor" that turns a **plan-as-code** into actions and a concise user-facing answer.

```

In [8]: # --- Helper: extract code between <execute_python>...</execute_python> ---
def _extract_execute_block(text: str) -> str:
    """
    Returns the Python code inside <execute_python>...</execute_python>.
    If no tags are found, assumes 'text' is already raw Python code.
    """
    if not text:
        raise RuntimeError("Empty content passed to code executor.")
    m = re.search(r"<execute_python>(.*?)</execute_python>", text, re.DOTALL)
    return m.group(1).strip() if m else text.strip()

# ----- 2) Code execution -----
def execute_generated_code(
    code_or_content: str,
    *,
    db,
    inventory_tbl,
    transactions_tbl,
    user_request: Optional[str] = None,
) -> Dict[str, Any]:
    """
    Execute code in a controlled namespace.
    Accepts either raw Python code OR full content with <execute_python> tag
    Returns minimal artifacts: stdout, error, and extracted answer.
    """
    # Extract code here (now centralized)
    code = _extract_execute_block(code_or_content)

    SAFE_GLOBALS = {
        "Query": Query,
        "get_current_balance": inv_utils.get_current_balance,
        "next_transaction_id": inv_utils.next_transaction_id,
        "user_request": user_request or "",
    }
    SAFE_LOCALS = {
        "db": db,
        "inventory_tbl": inventory_tbl,
        "transactions_tbl": transactions_tbl,
    }

    # Capture stdout from the executed code
    _stdout_buf, _old_stdout = io.StringIO(), sys.stdout
    sys.stdout = _stdout_buf
    err_text = None
    try:
        exec(code, SAFE_GLOBALS, SAFE_LOCALS)
    except Exception:
        err_text = traceback.format_exc()
    finally:
        sys.stdout = _old_stdout
    printed = _stdout_buf.getvalue().strip()

    # Extract possible answers set by the generated code
    answer = (

```

```

SAFE_LOCALS.get("answer_text")
or SAFE_LOCALS.get("answer_rows")
or SAFE_LOCALS.get("answer_json")
)

return {
    "code": code,           # <- ya sin etiquetas
    "stdout": printed,
    "error": err_text,
    "answer": answer,
    "transactions_tbl": transactions_tbl.all(), # For inspection
    "inventory_tbl": inventory_tbl.all(), # For inspection
}

```

You've checked the shelves and confirmed there's exactly one round style under \$100. Now the fun part: let's hand the model's plan-as-code to our executor and watch it do the work. The executor will peel out the `<execute_python>...</execute_python>` block, run it in a locked-down sandbox, and then show you everything that matters—what changed in the tables (before/after), any logs the plan printed, and the final, customer-friendly `answer_text`.

```

In [9]: # Execute the generated plan for the round-sunglasses question
result = execute_generated_code(
    full_content_round,           # the full LLM response you generated earlier
    db=db,
    inventory_tbl=inventory_tbl,
    transactions_tbl=transactions_tbl,
    user_request=prompt_round, # e.g., "Do you have any round sunglasses in
)

# Peek at exactly what Python the plan executed
utils.print_html(result["answer"], title="Plan Execution · Extracted Answer")

```

Plan Execution · Extracted Answer

Yes, we have our Classic sunglasses, a round frame, for \$60.

As you can see, this is the expected result based on our previous manual analysis.

2.4 Return Two Aviator Sunglasses

In the previous step we only **queried** the data, so inventory and transactions were unchanged.

Now let's handle a **return** scenario using the planning-in-code pattern:

Request: "Return 2 Aviator sunglasses I bought last week."

Before generating the plan, let's **inspect the current inventory** for the *Aviator* model.

```
In [10]: Item = Query()                                # Create a Query object to reference field

# Query: fetch all inventory rows whose 'name' is exactly "Aviator".
# Notes:
# - This is a case-sensitive equality check. "aviator" won't match.
# - If you need case-insensitive matching, consider a .test(...) or .matches
aviators = inventory_tbl.search(
    (Item.name == "Aviator")
)

# Display the matched documents in a readable JSON panel
utils.print_html(json.dumps(aviators, indent=2), title="Inventory status: Av
```

Inventory status: Aviator sunglasses before return

```
[
  {
    "item_id": "SG001",
    "name": "Aviator",
    "description": "Originally designed for pilots, these teardrop-shaped lenses with thin metal frames offer timeless appeal. The large lenses provide excellent coverage while the lightweight construction ensures comfort during long wear.",
    "quantity_in_stock": 23,
    "price": 80
  }
]
```

Inventory confirms one Aviator SKU in stock — **SG001 (Aviator): 23** units at **\$80** each.
Now let's generate a plan to answer the prompt:

```
In [11]: prompt_aviator = "Return 2 Aviator sunglasses I bought last week."

# Generate the plan-as-code (FULL content; may include <execute_python> tags
full_content_aviator = generate_llm_code(
    prompt_aviator,
    inventory_tbl=inventory_tbl,
    transactions_tbl=transactions_tbl,
    model="o4-mini",
    temperature=1,
)

# Inspect the LLM's plan + code (no execution here)
utils.print_html(full_content_aviator, title="Plan with Code (Full Response)
```

Plan with Code (Full Response)

```

<execute_python>
# 1. Imports and setup
from tinydb import Query
from datetime import datetime
import re

# 2. Initialize defaults
STATUS = None
answer_text = ""
# For logging
log_action = "mutate"
should_mutate = True

# 3. Parse user_request for intent, item, and quantity
# We're assuming `user_request` variable is provided in the env
# ironment.
match = re.search(r'\breturn\s+(\d+)\s+([A-Za-z ]+?)sunglasses', u
ser_request, re.IGNORECASE)
if not match:
    STATUS = "invalid_request"
    answer_text = "I can help with that—how many pairs would you l
ike to return?"
else:
    qty = int(match.group(1))
    item_name = match.group(2).strip()

    # 4. Query inventory for the requested item
    Item = Query()
    results = inventory_tbl.search(Item.name.test(lambda v, name=i
tem_name: v.lower() == name.lower(), item_name))

    if not results:
        # No such style
        STATUS = "no_match"
        # Suggest closest by price (cheapest in stock)
        available = inventory_tbl.all()
        # pick one with minimal price
        suggestion = min((i for i in available if i['quantity_in_s
tock'] > 0), key=lambda x: x['price'], default=None)
        if suggestion:
            answer_text = (f"We don't have {item_name} sunglasses,
but our {suggestion['name']} "
                           f"is available for ${suggestion['pric
e']}.")
        else:
            answer_text = "We don't have that style right now; ple
ase check back later or try another."
    else:
        item = results[0]
        # 5. Perform the return: update stock and create a transac
tion
        unit_price = item['price']

```



```

        line_total = unit_price * qty
        # Transaction amount is negative for a refund
        txn_amount = -line_total

        # Update stock
        new_stock = item['quantity_in_stock'] + qty
        inventory_tbl.update({'quantity_in_stock': new_stock}, Item.item_id == item['item_id'])

        # Create transaction record
        new_txn_id = next_transaction_id(transactions_tbl, prefix="TXN")
        prev_balance = get_current_balance(transactions_tbl)
        new_balance = prev_balance + txn_amount
        transactions_tbl.insert({
            'transaction_id': new_txn_id,
            'customer_name': match.group(0), # use full phrase as customer reference
            'transaction_summary': f"Return of {qty} {item_name} sunglasses",
            'transaction_amount': txn_amount,
            'balance_after_transaction': new_balance,
            'timestamp': datetime.now().isoformat()
        })

        STATUS = "success"
        answer_text = (f"Your return of {qty} {item_name} sunglasses is complete; "
                       f"we've refunded ${abs(txn_amount)}.")

    # 6. Logging
    print(f"LOG: ACTION={log_action} SHOULD_MUTATE={should_mutate} STATUS={STATUS}")

    # 7. answer_text and STATUS are set for the user
</execute_python>

```

Before we execute the plan, let's check the current status of the transactions.

```
In [12]: utils.print_html(json.dumps(transactions_tbl.all(), indent=2), title="Transa
```

Transactions Table Before Return

```
[
  {
    "transaction_id": "TXN001",
    "customer_name": "OPENING_BALANCE",
    "transaction_summary": "Daily opening register balance",
    "transaction_amount": 500.0,
    "balance_after_transaction": 500.0,
    "timestamp": "2026-08-05T18:52:21.280982"
  }
]
```

The transaction log currently shows a single entry — the opening balance (`TXN001`) for \$`500.00` recorded at `2025-10-03T09:16:59.628898` .

Ready to go—execute the plan by running the cell below.

```
In [13]: # Execute the generated plan for the round-sunglasses question
result = execute_generated_code(
    full_content_aviator,          # the full LLM response you generated ear
    db=db,
    inventory_tbl=inventory_tbl,
    transactions_tbl=transactions_tbl,
    user_request=prompt_aviator, # e.g., "Return 2 aviator sunglasses I boug
)

# Peek at exactly what Python the plan executed
utils.print_html(result["answer"], title="Plan Execution · Extracted Answer")
```

Plan Execution · Extracted Answer

Your return of 2 Aviator sunglasses is complete; we've refunded \$160.

You can see below that a new transaction has been inserted for the Aviator sunglasses return.

```
In [14]: utils.print_html(json.dumps(transactions_tbl.all(), indent=2), title="Transa
```

Transactions Table After Return

```
[
  {
    "transaction_id": "TXN001",
    "customer_name": "OPENING_BALANCE",
    "transaction_summary": "Daily opening register balance",
    "transaction_amount": 500.0,
    "balance_after_transaction": 500.0,
    "timestamp": "2026-08-05T18:52:21.280982"
  },
  {
    "transaction_id": "TXN002",
    "customer_name": "Return 2 Aviator sunglasses",
    "transaction_summary": "Return of 2 Aviator sunglasses",
    "transaction_amount": -160,
    "balance_after_transaction": 340.0,
    "timestamp": "2026-08-05T18:52:47.949344"
  }
]
```

And by running the cell below, you'll see the Aviator stock increase to 25
(`quantity_in_stock`).

```
In [15]: Item = Query()

aviators = inventory_tbl.search(
    (Item.name == "Aviator")
)

utils.print_html(json.dumps(aviators, indent=2), title="Inventory status: Av
```

Inventory status: Aviator sunglasses after return

```
[
  {
    "item_id": "SG001",
    "name": "Aviator",
    "description": "Originally designed for pilots, these teardrop-shaped lenses with thin metal frames offer timeless appeal. The large lenses provide excellent coverage while the lightweight construction ensures comfort during long wear.",
    "quantity_in_stock": 25,
    "price": 80
  }
]
```

3. Putting It All Together: Customer Service Agent

You've built the pieces—schema, prompt, code generator, and executor. Now let's wire them up into a single helper that takes a natural-language request, generates a plan-as-code, executes it safely, and shows the result (plus before/after tables).

What this agent does

- Optionally reseeds the demo data for a clean run.
- Generates the plan (Python inside `<execute_python>...</execute_python>`).
- Executes the plan in a controlled namespace (TinyDB + helpers).
- Surfaces a concise `answer_text` and renders before/after snapshots.

```
In [16]: def customer_service_agent(
    question: str,
    *,
    db,
    inventory_tbl,
    transactions_tbl,
    model: str = "o4-mini",
    temperature: float = 1.0,
    reseed: bool = False,
) -> dict:
    """
    End-to-end helper:
    1) (Optional) reseed inventory & transactions
    2) Generate plan-as-code from `question`
    3) Execute in a controlled namespace
    4) Render before/after snapshots and return artifacts

    Returns:
    {
        "full_content": <raw LLM response (may include <execute_python> tags>,
        "exec": {
            "code": <extracted python>,
            "stdout": <plan logs>,
            "error": <traceback or None>,
            "answer": <answer_text/rows/json>,
            "inventory_after": [...],
            "transactions_after": [...]
        }
    }
    """
    # 0) Optional reseed
    if reseed:
        inv_utils.create_inventory()
        inv_utils.create_transactions()

    # 1) Show the question
    utils.print_html(question, title="User Question")

    # 2) Generate plan-as-code (FULL content)
```

```

full_content = generate_llm_code(
    question,
    inventory_tbl=inventory_tbl,
    transactions_tbl=transactions_tbl,
    model=model,
    temperature=temperature,
)
utils.print_html(full_content, title="Plan with Code (Full Response)")

# 3) Before snapshots
utils.print_html(json.dumps(inventory_tbl.all(), indent=2), title="Inventory Before")
utils.print_html(json.dumps(transactions_tbl.all(), indent=2), title="Transactions Before")

# 4) Execute
exec_res = execute_generated_code(
    full_content,
    db=db,
    inventory_tbl=inventory_tbl,
    transactions_tbl=transactions_tbl,
    user_request=question,
)

# 5) After snapshots + final answer
utils.print_html(exec_res["answer"], title="Plan Execution · Extracted Answer")
utils.print_html(json.dumps(inventory_tbl.all(), indent=2), title="Inventory After")
utils.print_html(json.dumps(transactions_tbl.all(), indent=2), title="Transactions After")

# 6) Return artifacts
return {
    "full_content": full_content,
    "exec": {
        "code": exec_res["code"],
        "stdout": exec_res["stdout"],
        "error": exec_res["error"],
        "answer": exec_res["answer"],
        "inventory_after": inventory_tbl.all(),
        "transactions_after": transactions_tbl.all(),
    },
}

```

4. Try It Out (with the Customer Service Agent)

Use the `customer_service_agent(...)` helper to go from a natural-language request → plan-as-code → safe execution → before/after snapshots.

Try these prompts:

1. Read-only (Andrew's example):

"Do you have any round sunglasses in stock that are under \$100?"

2. Mutation — return:

"Return 2 Aviator sunglasses."

3. Mutation — purchase:

"Purchase 3 Wayfarer sunglasses for customer Alice."

4. Mutation - purchase multiple items: "I want to buy 3 pairs of classic sunglasses and 1 pair of aviator."

What does `reseed=True` do?

When you call `customer_service_agent(..., reseed=True)`, the agent *re-initializes* the demo data before running your prompt:

- **Resets** the `inventory_tbl` to the default product set.
- **Resets** the `transactions_tbl` to a single opening-balance entry.
- Ensures a **clean, reproducible** run so results aren't affected by previous tests.

Set `reseed=False` if you want to **preserve** the current state and continue from prior operations.

```
In [ ]: prompt = "I want to buy 3 pairs of classic sunglasses and 1 pair of aviator
out = customer_service_agent(
    prompt,
    db=db,
    inventory_tbl=inventory_tbl,
    transactions_tbl=transactions_tbl,
    model="o4-mini",
    temperature=1.0,
    reseed=True,    # set False to keep current state of the inventory and th
)
```

User Question

I want to buy 3 pairs of classic sunglasses and 1 pair of aviator sunglasses.

5. Takeaways

- **You let code be the plan.** Following Andrew's "code-as-action" idea, you had the model write Python that chains the steps (filter → compute → update) and then you just ran it.
- **You skipped the brittle tool soup.** Instead of piling on tiny tools or JSON plans, you used Python/TinyDB—giving the model a big, familiar toolbox that handles many query shapes with one prompt.

- **You kept runs safe and visible.** You executed in a controlled namespace, captured logs/errors, and reviewed before/after tables—so you always know what changed and why.



Congratulations!

You just finished the lab and built an *agentic* customer service workflow. You let the model write code as the plan, ran it safely, and used simple validations to keep updates reliable. When things failed, you surfaced clear, human-readable reasons; when things worked, you saw exactly what changed via before/after snapshots.

With this pattern—planning *in* code, plus transparent execution—you're ready to design your own workflows that feel automatic, safe, and easy to extend. 🚀

In []: